

Universidad Nacional Autónoma de México

Facultad de Ciencias

Cómputo Concurrente

2027-1

Práctica 03. Exclusión Mutua: Candados Clásicos y el Modelo de Memoria de Java (JMM)

ALUMNOs:

Martínez Hidalgo Paola Mildred - 319300217
Dominguez Sanchez Gustavo Alonso - 418047121

Fecha de entrega:
24 de septiembre de 2026

Profesor:
Gilde Valeria Rodríguez



Ejercicios

1. El algoritmo de candado Peterson solo funciona para 2 hilos, utiliza el algoritmo de candado Peterson para implementar un algoritmo de candado que funcione para 4 hilos. Hint: Apóyate del programa Peterson

   En cuanto a la implementación "DoublePeterson.java" se usó el pseudocódigo de la Tarea 2, en este caso como Peterson sólo funciona para dos hilos, lo que hicimos fue que tomamos nuestros cuatro hilos y los dividimos en dos grupos, así cada grupo sólo tendrá dos hilos, esos dos hilos se encontrarán en el mismo candado y competirán para ver qué hilo pasa, esto pasará en los dos candados. Después de que cada candado tiene un hilo que pasó estos dos hilos volverán a competir para que sólo uno pase a la sección crítica. Para saber qué hilo es se le asigna un único ID a cada uno, de esta manera sabemos qué hilo hizo qué tarea.

   - (a) Para probar que tu candado funciona, crea una implementación en donde utilices un ExecutorService para ejecutar 400 tareas, cada tarea debe aumentar en uno un objeto Contador.
     Utiliza tu candado para 4 hilos para tener consistencia en tu Contador.

     En este caso tenemos la clase llamada "TetsDoublePeterson.java", lo que se hace aquí es que prueba nuestra clase "DoublePeterson.java", en este caso le damos 400 tareas donde se va a trabajar exactamente con 4 hilos, lo que le estamos diciendo aquí a cada hilo es que tome el candado, incremente el contador y suelte el candado. Este nos tiene que devolver cuántas veces aumentó el contador cada hilo.

     Una salida que tuvimos fue la siguiente:

     ```
     Valor final del contador: 400
     Hilo id 0 incrementó 128 veces
     Hilo id 1 incrementó 149 veces
     Hilo id 2 incrementó 38 veces
     Hilo id 3 incrementó 85 veces
     ```

   - (b) De alguna forma obtén el número de veces que los hilos aumentan el contador. ¿Cada uno realiza exactamente 100 tareas o hay algunos que realizan más?

     Como vemos en la salida que pusimos en el inciso a, podemos ver que no fue equitativa la realización de tareas por hilo. En este caso ExecutorService no reparte las tareas una a una por hilo, lo que hace es que cada hilo va tomando la siguiente tarea de la fila en cuanto queda libre. Si un hilo tarda menos en pasar por el candado, agarra más tareas de la cola antes de que los otros terminen la suya.

     Así que contestando la pregunta no, cada hilo no realizó 100 tareas, hubo hilos que realizamos más, pero igual hubo otros que no llegaron ni a 50.

   - (c) ¿Consideras que la implementación cumple con Justicia? Justifica tu respuesta.

     Sí, en el sentido de que ningún hilo se quedó en cero, todos lograron entrar a la sección crítica múltiples veces, y el conteo final da exactamente 400. Pero no hay justicia en el sentido de "reparto equitativo".

     Adventencia: Puedes considerar el pseudocódigo de la Tarea 3 (DoubleP eterson). Solo ten cuidado con los id's, ya que si utilizas modulo 2 para Peterson y modulo 4 para DoublePeterson, puede pasar que dos hilos ejecuten un candado Peterson con el mismo id. Si dos hilos en Peterson tienen el mismo id entonces no se cumple exclusión mutua, y el candado DoublePeterson no cumplirá su función, no contará las 400 tareas.

2. En base a la implementación de Bakery vista en la clase teoría, implementa Bakery para 4 hilos.

   Hint: Crea los arreglos flag y label de tamaño 4, si consideras necesario utiliza campos volatile

   El programa `BakeryCuatroHilos` implementá el algoritmo Bakery visto en clase pero ahora adaptado estrictamente para 4 hilos. Aunque inicialmente el algoritmo se diseño arreglos `volatile` (no cambiando mucho al algoritmo de la clase) para las banderas `flag` y `label`, después nos dimos cuenta que para la JMM declararlo de esta forma solo garantiza la visibilidad y reordenamiento a la **referencia del arreglo**, NO a los elementos del arreglo, que es justo lo que necesitabamos. Esto provocaba los problemas antes mencionados, resultando en ciclos infinitos y congelando la ejecución.
   Para solucionarlo, sin utilizar la paquetería `Atomic`, la implementación propuesta expone a cada una de las

variables que se utilizar por cada hilo como `volatile` (`f0,f1,f2,f3,l0,l1,l2,l3`), en lugar del arreglo. De esta forma se fuerza a que cada lectura y escritura durante la fase de Doorwat y Waiting se haga directamente a la memoria principal (RAM) en lugar que la memoria local de cada hilo, garantizando que todos los hilos mantengan coherencia.

- (a) Con ayuda de un ExecutorService ejecuta 400 tareas, cada tarea debe aumentar en uno un objeto Contador. Utiliza tu candado para 4 hilos para tener consistencia en tu Contador.
  El programa de prueba `TestBakeryCuatroHilos` utiliza un `ExecutorService` con una pool fija de 4 hilos. Se creó una clase utilitaria (`ThreadID`) construida con `ThreadLocal` para asignar de manera segura a cada hilo de la pool un identificador. Este programa ejecuta 400 tareas concurrentes donde cada hilo hace uso del candado `BakeryCuatroHilos` para incrementar el contador compartido (`CounterNaive`). logrando procesar todas las tareas de forma segura. Con el uso de este candado, no existen condiciones de *Data Race*. En todas las pruebas realizadas, el programa realizó el total de tareas y finalizó correctamente.
- (b) De alguna forma obtén el número de veces que los hilos aumentan el contador. ¿Cada uno realiza exactamente 100 tareas o hay algunos que realizan más?
  Se muestran dos prubas de ejecución para `TestCakeryCuatroHilos`:

```
Valor final del contador: 400
Hilo id 0 incrementó 75 veces
Hilo id 1 incrementó 122 veces
Hilo id 2 incrementó 125 veces
Hilo id 3 incrementó 78 veces
```

```
Valor final del contador: 400
Hilo id 0 incrementó 102 veces
Hilo id 1 incrementó 101 veces
Hilo id 2 incrementó 99 veces
Hilo id 3 incrementó 98 veces
```

  Tal como se observa, en ninguna ejecución los hilos no realizan exactamente 100 tareas cada uno. Esto es de esperarse debido a que el sistema sigue siendo asíncrono, por lo que la velocidad en la que cada hilo ejecuta las tareas depende de varios factores como el planificador del sistema operativo o los tiempos de acceso a la memoria. Sin embargo, es importante resaltar que la distribución de tareas aunque no es exacta, **si es más equilibrada** respecto a las pruebas hechas con el cadado de Peterson, justamente por la naturaleza lógica de nuestro algoritmo Bakery.
- (c) ¿Consideras que la implementación cumple con Justicia? Justifica tu respuesta. Si cumple con Justicia y se garantiza mediante lo siguiente:
  - CUando un hilo entra al Doorway, calcula su turno tomando el máximo actual de los demás hilos y sumándole uno. Esto asegura que si un hilo pide su turno antes que otro, el primero tendrá un número menor y entrará primero a la sección crítica.
  - Si llegará a pasar (debido a la concurrencia), si dos hilos leen el mismo estado y calculan exactamente el mismo número de turno, el algoritmo utiliza el identificador del hilo (el ID) para desempatar, dándole prioridad al hilo con el ID de menor número.

  Por lo tanto es cumple con ser FIFO.

3. 3. Revisa el programa de Bakery que se les compartió como ejemplo, ejecútalo varias veces, revisa el código y contesta:

- (a) Revisa para que sirve el campo AtomicReference, y qué hacen los métodos compareAndSet() y get().
  Para explicar `AtomicReference`, podemos ver que en el código tenemos a `next`, el cual nos indica cuál es el siguiente nodo de la cola. Esto es importante porque `AtomicReference` permite que los diferentes hilos puedan consultar y modificar `next` de manera segura, evitando que dos o más hilos lo modifiquen al mismo tiempo y se genere un conflicto.
  El método `compareAndSet()` se puede entender como: "si tienes este valor, cámbialo por este nuevo valor". Si otro hilo ya modificó el valor, el cambio no se realiza y el método devuelve `false`. Esto ayuda a evitar que un hilo sobrescriba el trabajo que otro hilo acaba de realizar.
  El método `get()` simplemente permite consultar qué nodo está guardado actualmente en `next`.

- (b) Describe como funciona en a lo más 6 líneas de computadora.

  Para explicar cómo funciona, podemos retomar el ejemplo de la panadería. Cada persona toma un boleto y se forma al final de una fila. Los hilos intentan agregar su nodo (boleto) a la fila utilizando `compareAndSet()` para evitar conflictos con los demás hilos. Una vez que el nodo fue agregado, el hilo espera mientras las personas que están delante de él pasan. Cuando llega su turno, puede continuar y, al terminar, desactiva su bandera para permitir que avance el siguiente hilo. De esta manera, los hilos van entrando siguiendo el orden en el que se agregaron a la fila.

- (c) ¿Si next no es un AtomicReference sigue funcionando? Hint: Recuerda la Cola concurrente sin candados que implementaste en la Práctica 2

  No funcionaría de la misma manera. Si `next` no fuera un `AtomicReference`, ya no podríamos utilizar `compareAndSet()`. Esto permitiría que varios hilos intentaran modificar `next` al mismo tiempo sin tener una operación que garantice que sólo uno pueda realizar el cambio. Por lo tanto, podrían generarse conflictos entre los hilos y la cola dejaría de funcionar correctamente como una estructura concurrente sin candados.

- (d) ¿Consideras que mantiene la lógica de la implementación vista en clase/tu implementación del ejercicio anterior? Justifica porqué.

  Aunque ambas implementaciones buscan lo mismo (la exclusión mutua), la lógica es distinta.
  En nuestra implementación (`BakeryCuatroHilos`), el programa esta acotado por 4 hilos, y la lógica se basa en generar y asignar turnos mientras se atiende al que tenga el número más bajo.
  Por otro lado en `Bakery` el algoritmo no está acotado, ya que se basa en una lista enlazada. No hay números que indiquen el turno. En su lugar, el orden se establece justamente por la misma lista ligada (las tareas se realizan de acuerdo al orden en el que se van añadiendo).